Open in app ↗

Search

This member-only story is on us. Upgrade to access all of Medium.

Member-only story

Tooltips with Python's Matplotlib

How to add tooltips to your Matplotlib charts

Thiago Carvalho · Follow

Published in Towards Data Science

6 min read · May 27, 2021

Listen Share ••• More

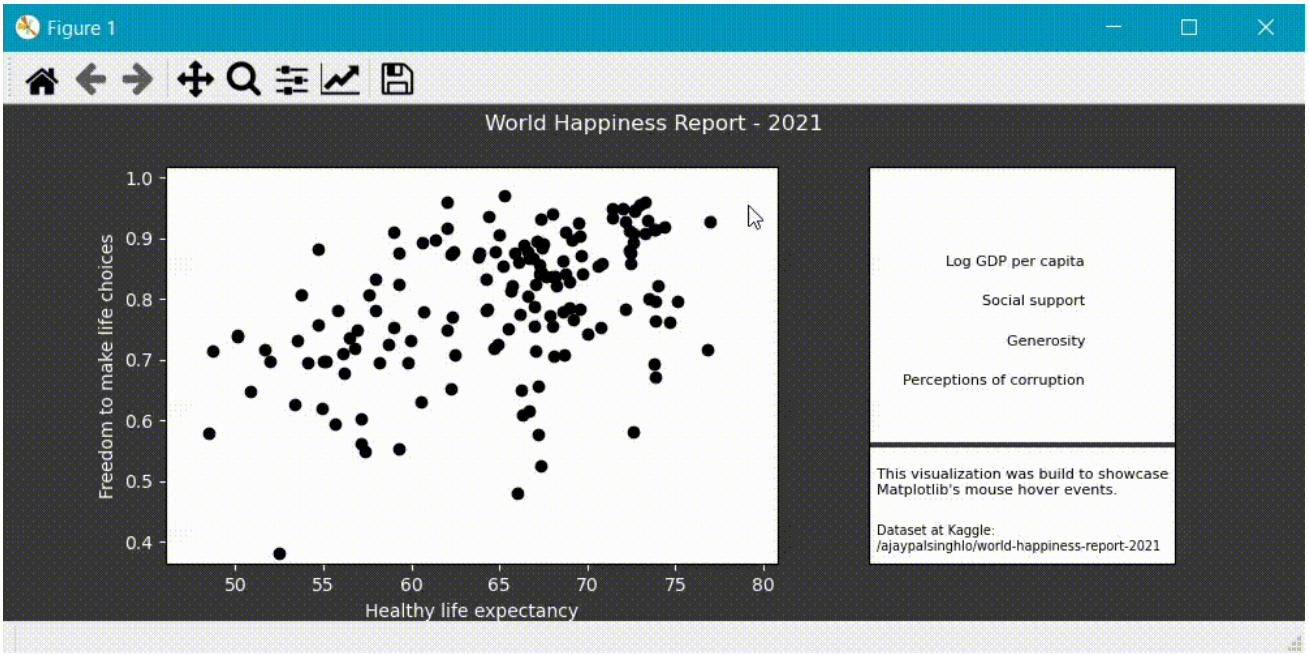


Image by author

Allowing the user to hover the mouse or click a part of our visualization to get more context is excellent. And tooltips are a perfect way to save space in our chart.

For example, for most scatter plots labelling every point can get tricky. You can color-code them and use a legend, but even that can get messy when there's too

much data.

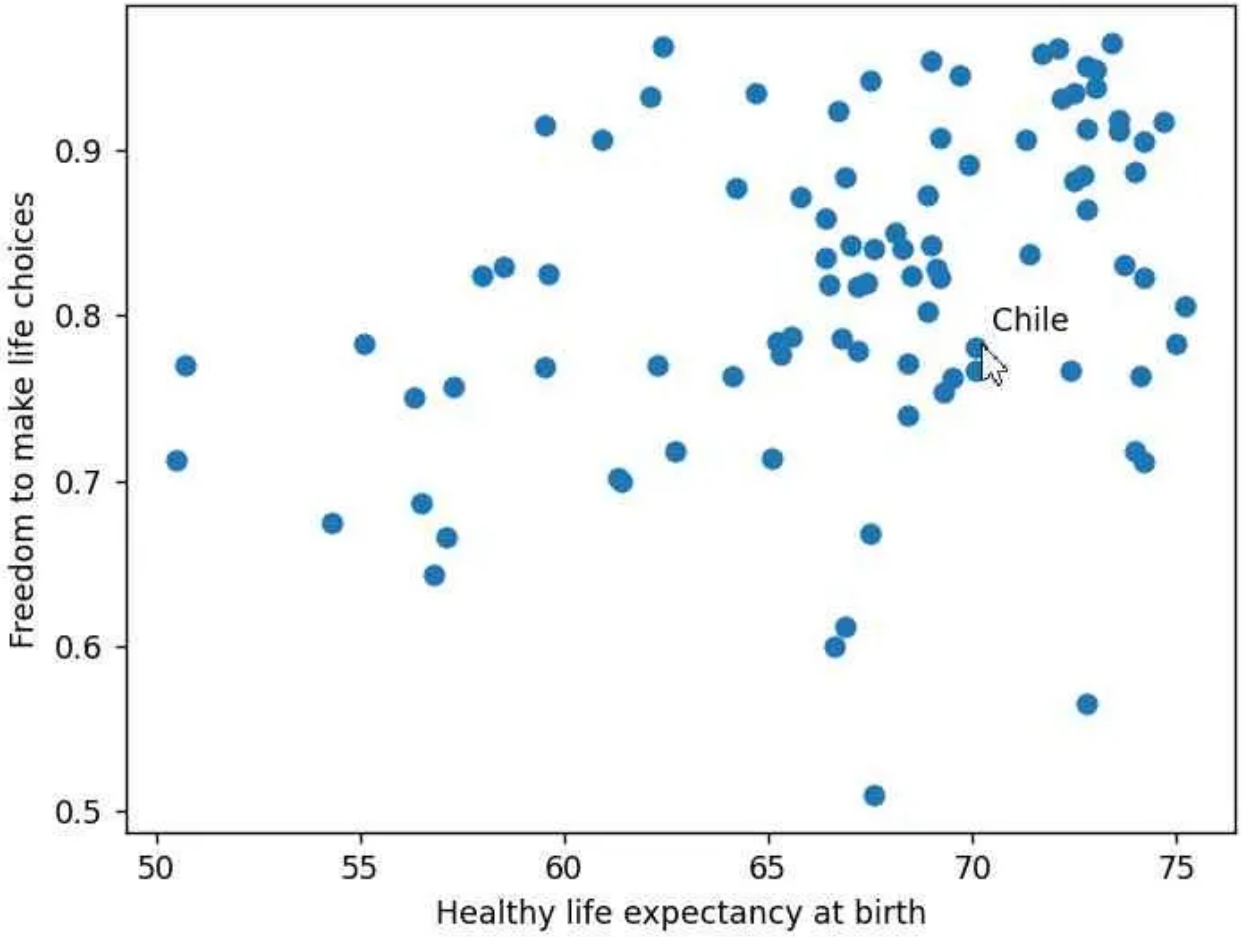


Image by author

In this article, we'll get a look at how to use event handlers and add tooltips to Matplotlib charts.

. . .

mpl_connect

For the following examples, we'll use Matplotlib and Pandas. The data comes from the World Happiness Report. You can find a .xls on their website as 'Data Panel' or get a .csv from Kaggle.

I'm using Jupyter Lab, so I need the magic command `%matplotlib widget` to correctly render the figure.

You may need a different command such as `%matplotlib notebook` or `%matplotlib`

`ipympl` depending on your version of Jupyter. If you're running a script, you don't need anything.

```
import matplotlib.pyplot as plt
import pandas as pd
%matplotlib widget

# https://www.kaggle.com/ajaypalsinghlo/world-happiness-report-2021
df = pd.read_csv('data/world-happiness-report.csv')
df = df[df['year'] == 2020]

x_name = 'Healthy life expectancy at birth'
y_name = 'Freedom to make life choices'
tooltip_name = 'Country name'

x = df[x_name]
y = df[y_name]
tt = df[tooltip_name].values

df
```

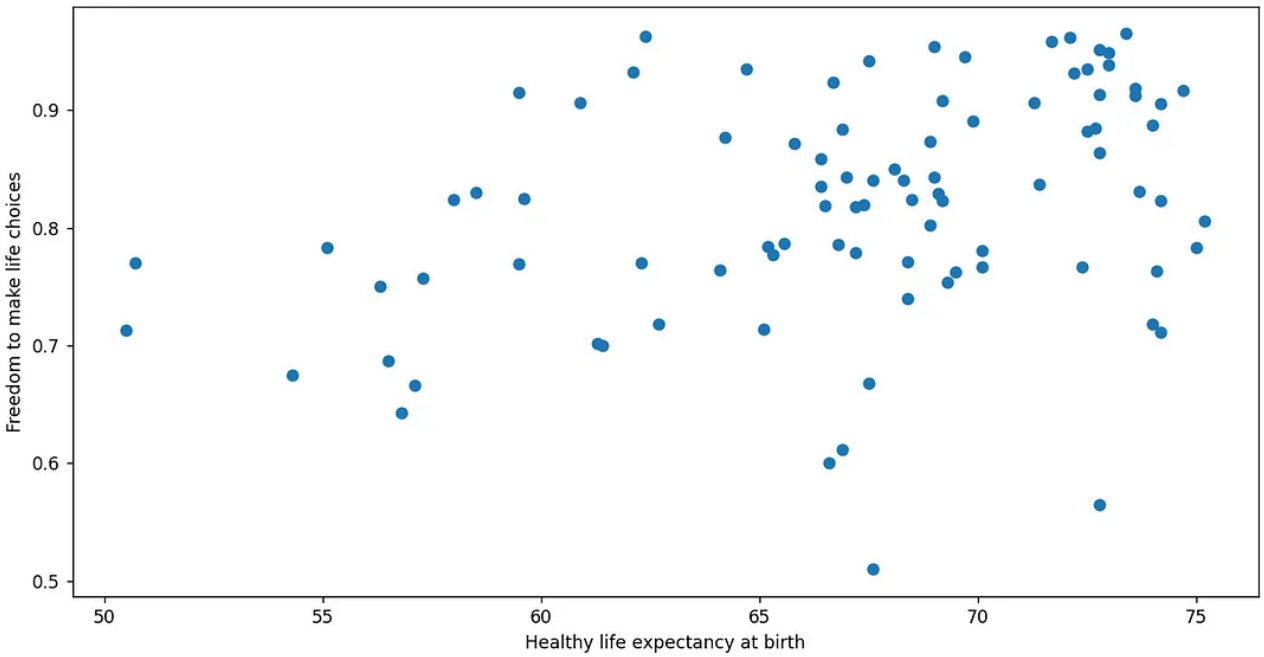
	Country name	year	Life Ladder	Log GDP per capita	Social support	Healthy life expectancy at birth	Freedom to make life choices	Generosity	Perceptions of corruption	Positive affect	Negative affect
24	Albania	2020	5.365	9.497	0.710	69.3	0.754	0.007	0.891	0.679	0.265
51	Argentina	2020	5.901	9.850	0.897	69.2	0.823	-0.122	0.816	0.764	0.342
79	Australia	2020	7.137	10.760	0.937	74.2	0.905	0.210	0.491	0.769	0.205
92	Austria	2020	7.213	10.851	0.925	73.6	0.912	0.011	0.464	0.769	0.206
117	Bahrain	2020	6.173	10.620	0.848	69.7	0.945	0.132	NaN	0.790	0.297
...	...	...	...	...	...	...	...	...	...	...	...
1850	United States	2020	7.028	11.001	0.937	68.1	0.850	0.034	0.678	0.787	0.295
1865	Uruguay	2020	6.310	9.937	0.921	69.2	0.908	-0.084	0.491	0.807	0.265
1893	Venezuela	2020	4.574	NaN	0.805	66.9	0.612	NaN	0.811	0.722	0.396
1933	Zambia	2020	4.838	8.117	0.767	56.3	0.750	0.056	0.810	0.691	0.345
1948	Zimbabwe	2020	3.160	7.829	0.717	56.8	0.643	-0.009	0.789	0.703	0.346

95 rows × 11 columns

Image by author

Let's start with a simple scatter plot and build from it.

```
fig, ax = plt.subplots(1, figsize=(12,6))

ax.scatter(x, y)

plt.xlabel(x_name)
plt.ylabel(y_name)

plt.show()
```



Simple scatter plot — Image by author

Now we'll create a function that'll be called when hovering our visualization.

To start simple, we'll add a text saying 'Chart Ready' and our function will change this text to blank when called.

```
fig, ax = plt.subplots(1, figsize=(12,6))

ax.scatter(x,y)
plt.xlabel(x_name)
plt.ylabel(y_name)

annot_x = (plt.xlim()[1] + plt.xlim()[0])/2
annot_y = (plt.ylim()[1] + plt.ylim()[0])/2
txt = ax.text(annot_x, annot_y, "Chart Ready",
              ha='center', fontsize=36, color='#DD4012')

def hover(event):
    txt.set_text("")

fig.canvas.mpl_connect("motion_notify_event", hover)

plt.show()
```

Detecting cursor movement — Image by author

We're using Matplotlib's "**motion_notify_event**" to detect the mouse movement, but there are plenty of other events you can try.

> **Cursor XY coordinates**
> The idea is to draw the chart with all the elements we'll use and then use the events to modify those elements.

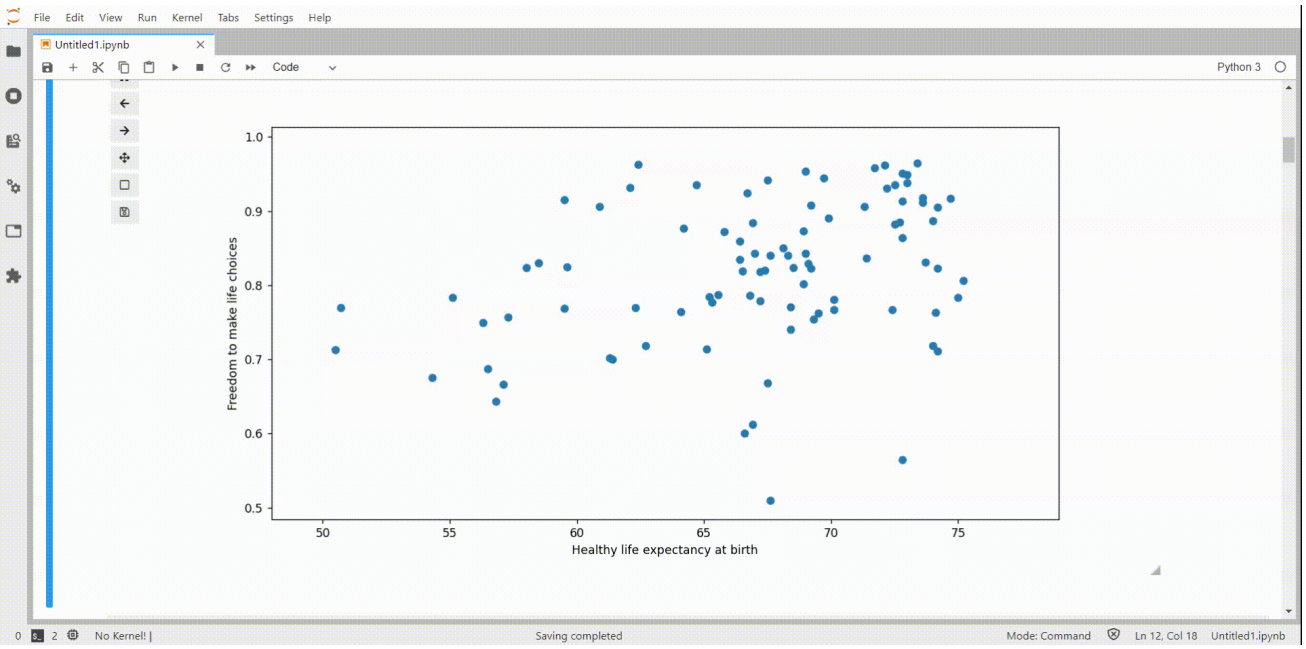
button_press_event, button_release_event, draw_event, key_press_event, key_release_event, pick_event, resize_event, scroll_event, figure_enter_event, figure_leave_event, axes_enter_event, axes_leave_event, close_event.

So even if some element is only displayed when a specific event is triggered, we still should define it outside of our function.

The event we receive from mpl_connect at our hover function has some valuable properties; let's try to get the XY coordinates of the mouse.

```
plt.close('all')
fig, ax = plt.subplots(1, figsize=(12,6))

ax.scatter(x,y)
plt.xlabel(x_name)
plt.ylabel(y_name)

lnx = plt.plot([60,60], [0,1.5], color='black', linewidth=0.3)
lny = plt.plot([0,100], [1.5,1.5], color='black', linewidth=0.3)
lnx[0].set_linestyle('None')
lny[0].set_linestyle('None')

plt.xlim(x.min()*0.95, x.max()*1.05)
plt.ylim(y.min()*0.95, y.max()*1.05)
```

```
def hover(event):
    lnx[0].set_data([event.xdata, event.xdata], [0, 1.5])
    lnx[0].set_linestyle('--')
    lny[0].set_data([0,100], [event.ydata, event.ydata])
    lny[0].set_linestyle('--')

fig.canvas.mpl_connect("motion_notify_event", hover)

plt.show()
```



Drawing lines with the mouse position — Image by author

We added the lines, set their style to none, and when the cursor motion is detected, we change the data and style of that line to display it.

. . .

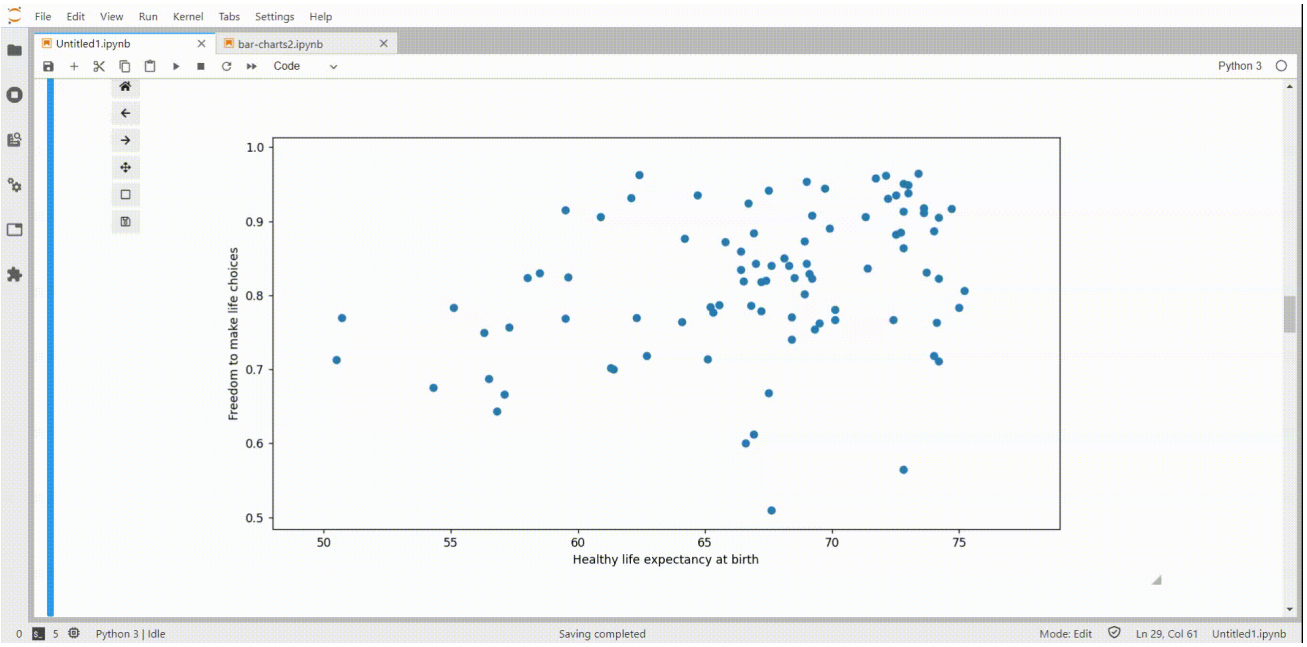
Tooltips

Great! We know how to add and modify elements in our plot and detect the movement of the cursor.

We'll create a blank annotation and check if the mouse position is over one of the plotted points. When it is, we change the text, position, and visibility of the annotation accordingly.

```
plt.close('all')
```

```
fig, ax = plt.subplots(1, figsize=(12,6))

# plot and labels
sc = ax.scatter(x,y)
plt.xlabel(x_name)
plt.ylabel(y_name)

# cursor grid lines
lnx = plt.plot([60,60], [0,1.5], color='black', linewidth=0.3)
lny = plt.plot([0,100], [1.5,1.5], color='black', linewidth=0.3)
lnx[0].set_linestyle('None')
lny[0].set_linestyle('None')

# annotation
annot = ax.annotate("", xy=(0,0), xytext=(5,5),textcoords="offset
points")
annot.set_visible(False)

# xy limits
plt.xlim(x.min()*0.95, x.max()*1.05)
plt.ylim(y.min()*0.95, y.max()*1.05)

def hover(event):
    # check if event was in the axis
    if event.inaxes == ax:
        # draw lines and make sure they're visible
        lnx[0].set_data([event.xdata, event.xdata], [0, 1.5])
        lnx[0].set_linestyle('--')
        lny[0].set_data([0,100], [event.ydata, event.ydata])
        lny[0].set_linestyle('--')
        lnx[0].set_visible(True)
        lny[0].set_visible(True)

        # get the points contained in the event
        cont, ind = sc.contains(event)
        if cont:
            # change annotation position
            annot.xy = (event.xdata, event.ydata)
            # write the name of every point contained in the event
            annot.set_text("{}".format(', '.join([tt[n] for n in
ind["ind"]])))
            annot.set_visible(True)
        else:
            annot.set_visible(False)
    else:
        lnx[0].set_visible(False)
        lny[0].set_visible(False)

fig.canvas.mpl_connect("motion_notify_event", hover)

plt.show()
```



Tooltips! — Image by author

There it is!

We have a tooltip on our chart, but it wasn't exactly straightforward to build. We're drawing a scatter plot that returns a PathCollection object with the convenient method `.contains()`.

Matplotlib returns different objects for different visualizations **.bar** will return BarContainer, **.plot** will return a list of Line2D, and so on.

That means we'll have to develop a different solution to add tooltips for each visualization.

The freedom to develop how your chart will behave for each event is excellent. But depending on how complex your visualization is, this might test your patience.

. . .

Luckily there are other solutions for adding tooltips to Matplotlib.

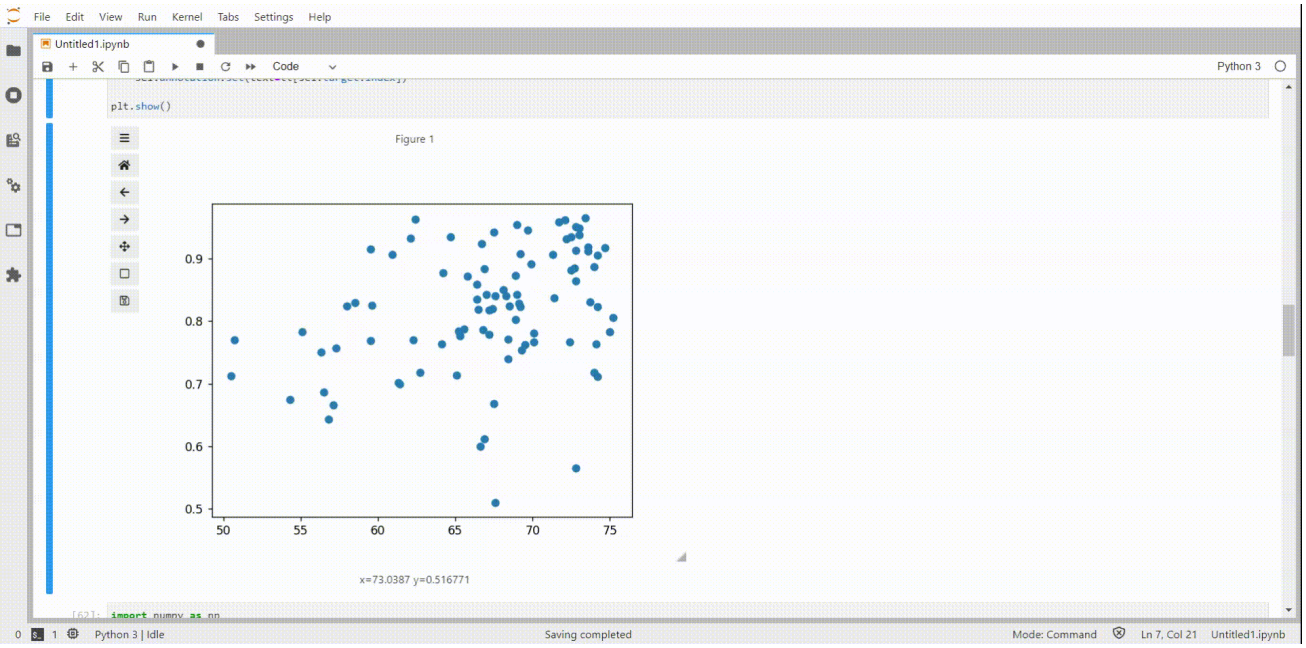
Mplcursors

This handy package facilitates mouse events in Matplotlib with some easy-to-use methods.

They also have lots of examples that are easy to customize. So instead of digging Matplotlib's documents and trying to figure out your way. You can replace a

couple of lines from the examples and get your tooltips in no time.

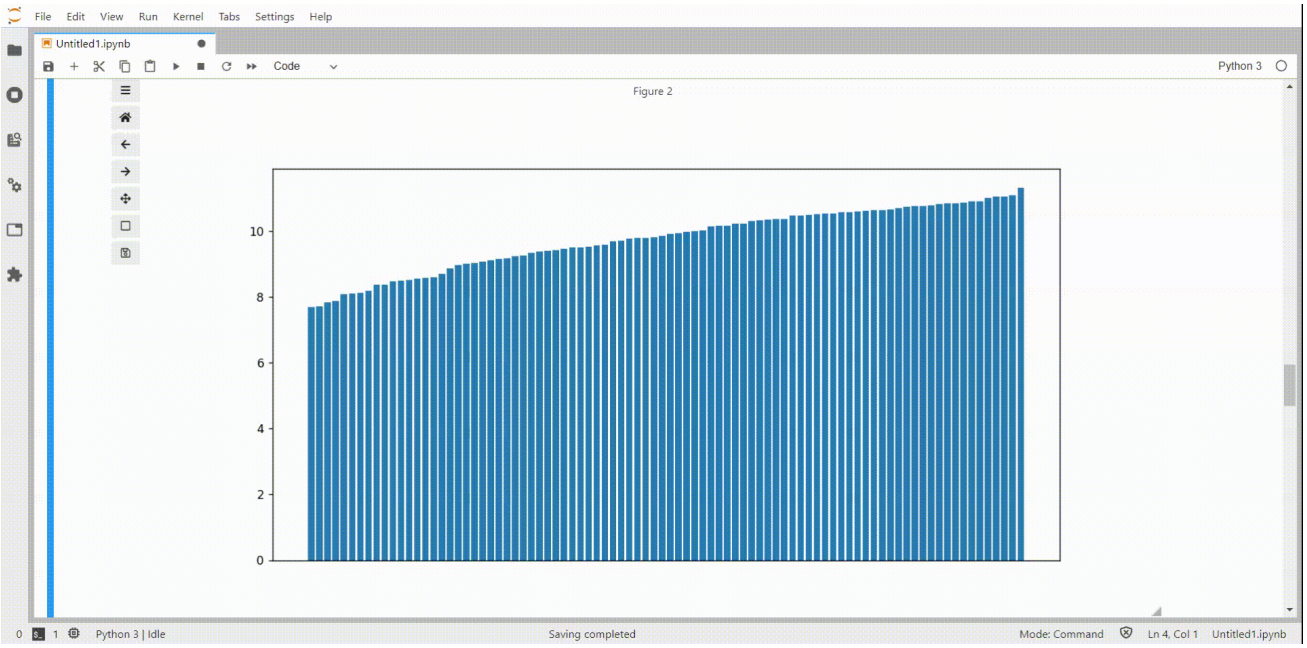
```
import matplotlib.pyplot as plt
import mplcursors
%matplotlib widget

fig, ax = plt.subplots()
sc = ax.scatter(x,y)

# by default the tooltip is displayed "onclick"
# we can change it by setting hover to True
cursor = mplcursors.cursor(sc, hover=True)

# by default the annotation displays the xy positions
# this is to change it to the countries name
@cursor.connect("add")
def on_add(sel):
    sel.annotation.set(text=tt[sel.target.index])

plt.show()
```



Easier Tooltips — Image by author

```
df = df.sort_values('Log GDP per capita')
df = df[df['Log GDP per capita'].notna()]

fig, ax = plt.subplots(figsize=(12,6))
ax.bar(df['Country name'], df['Log GDP per capita'],
align="center")
plt.xticks([])

cursor = mplcursors.cursor(hover=mplcursors.HoverMode.Transient)
```

```
@cursor.connect("add")
def on_add(sel):
    sel.annotation.set(text=df.iloc[sel.target.index]['Country name'])

plt.show()
```

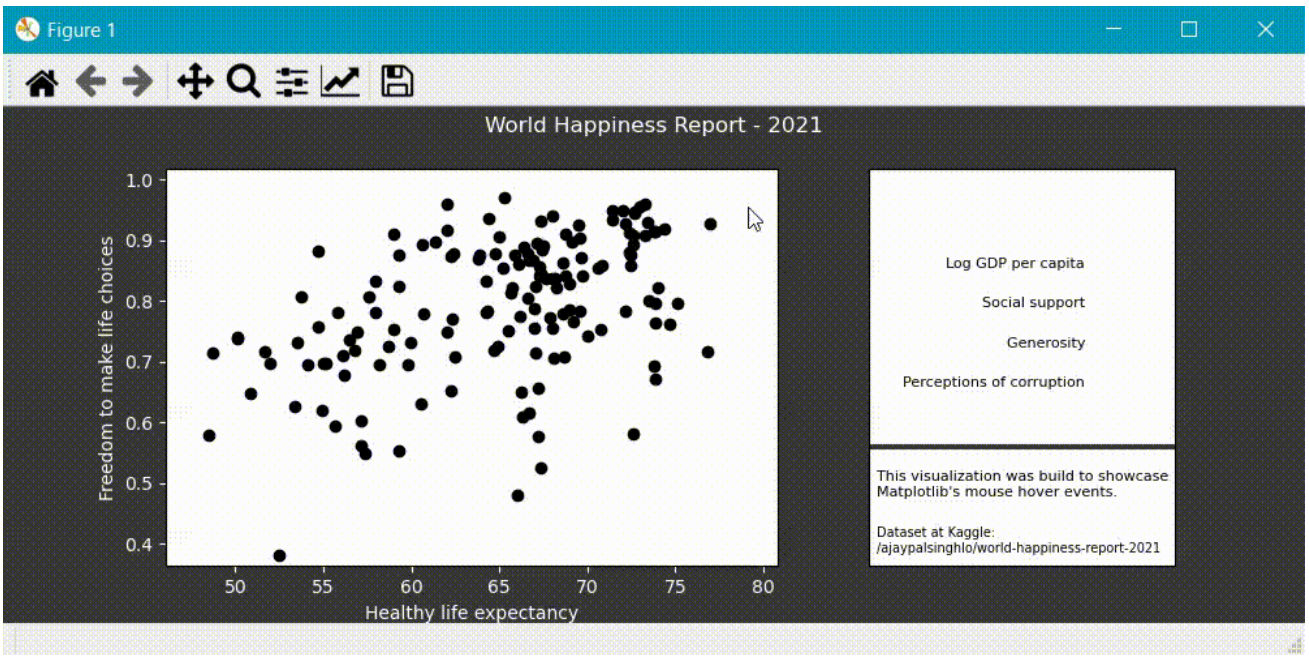


Tooltips in a bar chart — Image by author

. . .

Conclusion

Overall, adding tooltips to Matplotlib charts can quickly get complex and even with Mplcursors, it isn't as easy as it could be.



Code — Image by author

The way I see it is, we can get around with Mplcursors to add some extra functionalities to your visualization without struggling so much. If you are willing to put some more effort into it, you can build the most complex visualizations with all the available events Matplotlib has.

There are other options with Python, such as Plotly for designing interactive data visualizations; or MplD3, which adds some extra functionalities to Matplotlib charts by converting them into D3.js.

. . .

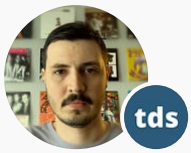
Thanks for reading!

All images are my own. Feel free to use them.

References:

https://matplotlib.org/stable/users/event_handling.html;
https://matplotlib.org/stable/api/backend_bases_api.html;
https://matplotlib.org/stable/api/_as_gen/matplotlib.lines.Line2D.html;
https://matplotlib.org/stable/api/_as_gen/matplotlib.pyplot.bar.html;
https://mplcursors.readthedocs.io/en/stable/;

Data Visualization | Python | Matplotlib | Data Analysis | Tooltips



Follow

Written by Thiago Carvalho